

LUMINAR TECHNO LAB  
Trivandrum

---

# Plant Pathology Prediction using CNN:

## With Augmentation, Without Augmentation, and Transfer Learning

---

*A Project Report on Convolutional Neural Networks*  
*Submitted in fulfilment of the CNN-Based Project Assessment*

Submitted by  
**Abhijith A B**

Module: AI & Machine Learning — CNN Training  
Assessment Type: Individual Practical Implementation & Report  
Case Study B — Plant Pathology 2020 (FGVC7)  
August 2026

# 1. Introduction

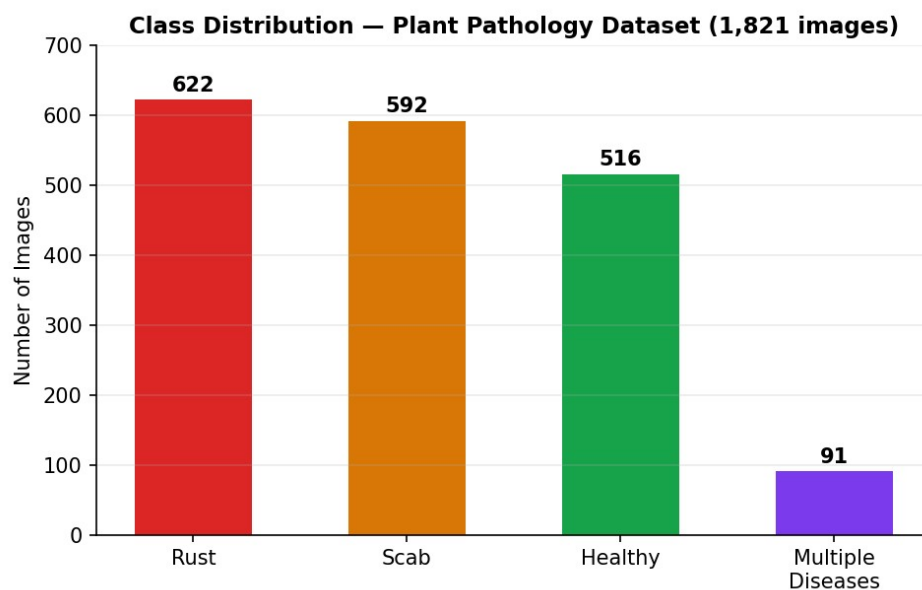
Apple orchards lose a meaningful share of their yield every year to foliar diseases that are hard to catch early by eye. This project builds an image classifier that looks at a photograph of an apple leaf and predicts whether it is healthy, or affected by rust, scab, or a combination of diseases. The dataset used is the Plant Pathology 2020 (FGVC7) dataset, released for a CVPR 2020 workshop, which contains real photographs taken in orchard conditions rather than lab-controlled shots — meaning the images carry natural variation in lighting, angle, and background.

The objective is to compare three different Convolutional Neural Network (CNN) approaches on this task: a custom CNN trained without any augmentation (Model 1), the same architecture trained with augmented data (Model 2), and a transfer-learning approach built on EfficientNetB0 (Model 3). The first two models have been trained and evaluated; Model 3 has its architecture defined and its base network prepared, but training has not been run yet at the time of this report — this is documented in Section 8.

## 2. Dataset Description

The dataset consists of 1,821 labelled leaf images, each associated with one of four classes: healthy, rust, scab, or multiple\_diseases. Labels were originally provided as a one-hot encoded CSV (train.csv), which was collapsed back into a single class column for this project using the column with the maximum value per row.

| Class             | Image Count | Share | Label Index |
|-------------------|-------------|-------|-------------|
| Rust              | 622         | 34.2% | 2           |
| Scab              | 592         | 32.5% | 3           |
| Healthy           | 516         | 28.3% | 0           |
| Multiple Diseases | 91          | 5.0%  | 1           |



*Figure 1. Class distribution across the 1,821 original training images.*

The class distribution is noticeably imbalanced: `multiple_diseases` makes up only about 5% of the dataset, while the other three classes are each in the 500–620 image range. This imbalance is carried through to both the training and test splits (the split was stratified specifically to preserve it, see Section 3), and it is worth keeping in mind when interpreting the per-class reliability of the model — the minority class has far fewer examples for the network to learn from.

### 3. Data Preprocessing

Each image was read from disk using OpenCV (`cv2.imread`) and resized to a fixed spatial size before being stacked into a single NumPy array. Two different target sizes were used at different stages of the project: 100×100 pixels for the two custom CNN models (Model 1 and Model 2), and 224×224 pixels when the pipeline was re-run to prepare data for the EfficientNetB0-based transfer learning model, since EfficientNetB0 expects a 224×224 input by design.

One implementation detail worth flagging honestly: OpenCV loads images in BGR channel order by default, and the code does not explicitly convert this to RGB before training. This does not affect the custom CNN models, since they learn directly from whatever channel order they are given — but it is worth correcting before Model 3 is trained, since EfficientNetB0's pretrained ImageNet weights were learned on RGB images, and feeding it BGR data (even after applying `preprocess_input`) could quietly reduce the benefit of the pretrained features.

For normalisation, the active pipeline in the notebook uses Keras's EfficientNet-specific `preprocess_input` function, applied after converting the image array to float32. This function rescales and mean-centres pixel values according to the statistics EfficientNet's ImageNet-pretrained weights expect, rather than a plain divide-by-255 scaling. This choice was made because the same preprocessed array is reused for all three models in this project — keeping one consistent preprocessing pipeline avoids maintaining two separate data pipelines and makes the eventual transfer-learning comparison more apples-to-apples. (Earlier, simpler versions of the pipeline — a plain `/255` rescale — were tried and left in the notebook as commented-out code, which is why they appear there but are not the active path.)

Converting the data to float32 is necessary for gradient-based training, since the model's weight updates rely on continuous-valued gradients rather than integer pixel values — but it does come at a real memory cost. The fully augmented dataset (10,926 images at 224×224×3) occupies roughly 6.27 GB in float32, which is a meaningful number to be aware of when working in a memory-limited environment such as Google Colab's free tier.

Finally, the data was split into training and test sets using an 80/20 stratified split (`train_test_split`, `random_state=42`, stratify on class label), which preserves the same class ratios described in Section 2 in both the training and test sets. A separate validation set was not carved out; instead, the test set doubles as the validation set passed to `model.fit()` for monitoring during training. This is a reasonable simplification for a project of this scope, though it means the test accuracy reported in Sections 5 and 6 is technically the same data the EarlyStopping callback used to pick the best epoch — a nuance worth being upfront about rather than treating the test numbers as being from a fully untouched holdout.

## 4. Data Augmentation

Rather than using Keras's built-in ImageDataGenerator, augmentation in this project was implemented manually with OpenCV, as five separate functions, each applied once to every original image. This means the augmentation is deterministic and pre-computed — every original image produces exactly five augmented variants, which are generated once and then concatenated with the originals, rather than being randomly regenerated on the fly each epoch as ImageDataGenerator would do.

| Technique              | Parameters Used                                              | Purpose                                                                                    |
|------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Rotation</b>        | $\pm 15^\circ$ , reflect-padded borders                      | Robustness to camera angle when the leaf was photographed                                  |
| <b>Horizontal Flip</b> | Mirror on vertical axis                                      | Leaves have no fixed left-right orientation, so this is a safe, label-preserving transform |
| <b>Zoom</b>            | Scale factor $0.8\times - 1.2\times$ , centre-cropped/padded | Simulates the leaf being photographed from different distances                             |
| <b>Shift</b>           | $\pm 10$ px, both axes, reflect-padded                       | Simulates imperfect centring/framing of the leaf in the shot                               |
| <b>Brightness</b>      | Scale factor $0.7\times - 1.3\times$ , clipped to $[0,255]$  | Simulates varying orchard lighting and time-of-day conditions                              |

Applying all five techniques to every one of the 1,821 original images produced 9,105 augmented images, which combined with the originals gives a total of 10,926 images — a roughly  $6\times$  increase in training data. It is worth noting that this augmentation multiplies every class by the same factor, so it increases the absolute number of examples in the minority multiple\_diseases class but does not correct its proportional under-representation relative to the other three classes.

## 5. Model 1 — Custom CNN (No Augmentation)

Model 1 is a CNN built from scratch (i.e. not pretrained) using the Keras Sequential API, trained only on the 1,821 original images before any augmentation was introduced. It serves as the baseline against which the benefit of augmentation (Model 2) and transfer learning (Model 3) can be measured.

### 5.1 Architecture and Design Rationale

The network follows a fairly standard three-block convolutional design, where each block doubles the number of filters while halving the spatial resolution. The reasoning behind each layer type used is explained below, since the brief specifically asks for this to be justified rather than just listed.

- Conv2D layers slide a small learnable filter ( $3\times 3$  here) across the image to detect local patterns — edges, colour blotches, texture changes — which are the earliest and most informative visual cues for something like a rust pustule or a scab lesion. Stacking three such blocks ( $32 \rightarrow 64 \rightarrow 128$  filters) lets the network build from simple edges in the first block to more disease-specific texture combinations in the third.
- Batch Normalisation, placed right after each Conv2D layer, normalises the activations coming out of that layer across the current batch. In practice this stabilises training, reduces sensitivity to weight initialisation, and allows a comparatively higher learning rate without the optimiser

diverging — it is one of the main reasons this network could be trained in a small number of epochs.

- ReLU activation was chosen over sigmoid or tanh mainly because it does not saturate for positive inputs, which keeps gradients from vanishing as they are back-propagated through several stacked layers, and because it is computationally cheap (a simple threshold at zero).
- MaxPooling2D (2×2) after every block halves the height and width of the feature map, which both reduces the number of computations and parameters in the following layers and gives the network a small amount of translation invariance — a disease spot slightly off-centre in the frame still activates the same downstream features.
- Flatten converts the final 12×12×128 feature map into a single 18,432-length vector so that it can be passed into the dense classification head.
- Dense(128, ReLU) combines the flattened spatial features into a smaller, higher-level representation before the final classification step.
- Dropout(0.2) randomly zeroes 20% of the units in this dense layer during training, which discourages the network from relying too heavily on any single neuron and is one of the standard defences against overfitting — a real concern here given that only 1,821 images are available to train on.
- The final Dense(4, Softmax) layer outputs a probability distribution across the four disease classes. Softmax is the appropriate choice here (rather than, say, four independent sigmoids) because the classes are mutually exclusive — a leaf is assigned to exactly one of the four categories, and the four output probabilities are constrained to sum to 1.

| Layer (type)                   | Output Shape         | Parameters |
|--------------------------------|----------------------|------------|
| conv2d (Conv2D, 32 filters)    | (None, 100, 100, 32) | 896        |
| batch_normalization            | (None, 100, 100, 32) | 128        |
| activation (ReLU)              | (None, 100, 100, 32) | 0          |
| max_pooling2d (2×2)            | (None, 50, 50, 32)   | 0          |
| conv2d_1 (Conv2D, 64 filters)  | (None, 50, 50, 64)   | 18,496     |
| batch_normalization_1          | (None, 50, 50, 64)   | 256        |
| activation_1 (ReLU)            | (None, 50, 50, 64)   | 0          |
| max_pooling2d_1 (2×2)          | (None, 25, 25, 64)   | 0          |
| conv2d_2 (Conv2D, 128 filters) | (None, 25, 25, 128)  | 73,856     |
| batch_normalization_2          | (None, 25, 25, 128)  | 512        |
| activation_2 (ReLU)            | (None, 25, 25, 128)  | 0          |
| max_pooling2d_2 (2×2)          | (None, 12, 12, 128)  | 0          |
| flatten                        | (None, 18432)        | 0          |
| dense (128, ReLU)              | (None, 128)          | 2,359,424  |

| Layer (type)         | Output Shape | Parameters |
|----------------------|--------------|------------|
| dropout (rate = 0.2) | (None, 128)  | 0          |
| dense_1 (4, Softmax) | (None, 4)    | 516        |

### Parameter Summary

|                                 |                     |
|---------------------------------|---------------------|
| <b>Total parameters</b>         | 2,454,084 (9.36 MB) |
| <b>Trainable parameters</b>     | 2,453,636 (9.36 MB) |
| <b>Non-trainable parameters</b> | 448 (1.75 KB)       |

## 5.2 Training Configuration

The model was compiled with the Adam optimiser and categorical cross-entropy loss, which is the standard pairing for one-hot encoded, multi-class classification problems, and trained with an EarlyStopping callback so that training would not continue past the point where the model stopped generalising.

| Hyperparameter              | Setting                   | Notes                                                        |
|-----------------------------|---------------------------|--------------------------------------------------------------|
| <b>Optimiser</b>            | Adam                      | Adaptive learning rate; good default for image CNNs          |
| <b>Loss function</b>        | Categorical Cross-Entropy | Standard for one-hot, mutually-exclusive multi-class targets |
| <b>Batch size</b>           | 32                        | Balances gradient stability and memory on Colab GPU          |
| <b>Epochs (max)</b>         | 20                        | Upper bound; EarlyStopping may cut this short                |
| <b>Callback</b>             | EarlyStopping             | monitor = 'val_loss'                                         |
| <b>Patience</b>             | 8 epochs                  | Training stops if val_loss does not improve for 8 epochs     |
| <b>Restore best weights</b> | True                      | Final model = weights from the best (lowest val_loss) epoch  |
| <b>Validation data</b>      | Held-out test split       | validation_data = (x_test, y_test)                           |

## 5.3 Training and Validation Curves

Training accuracy climbed smoothly and continuously across all 20 epochs, reaching 92.45% with a training loss of 0.19. Validation accuracy told a very different story: it hovered around 33–40% for the first nine

epochs, then became increasingly volatile, briefly touching 78.4% at epoch 17 before drifting back down to 67.7% by epoch 20. Validation loss followed a similarly erratic path, spiking sharply to 4.24 at epoch 13 even as training loss kept falling.

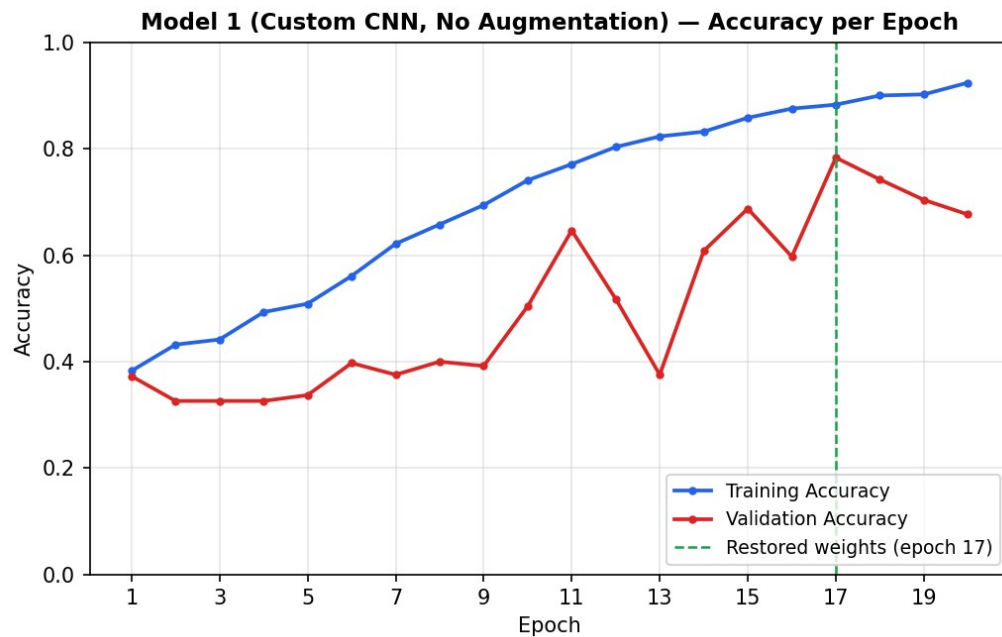


Figure 2. Model 1 — Training vs. validation accuracy per epoch.

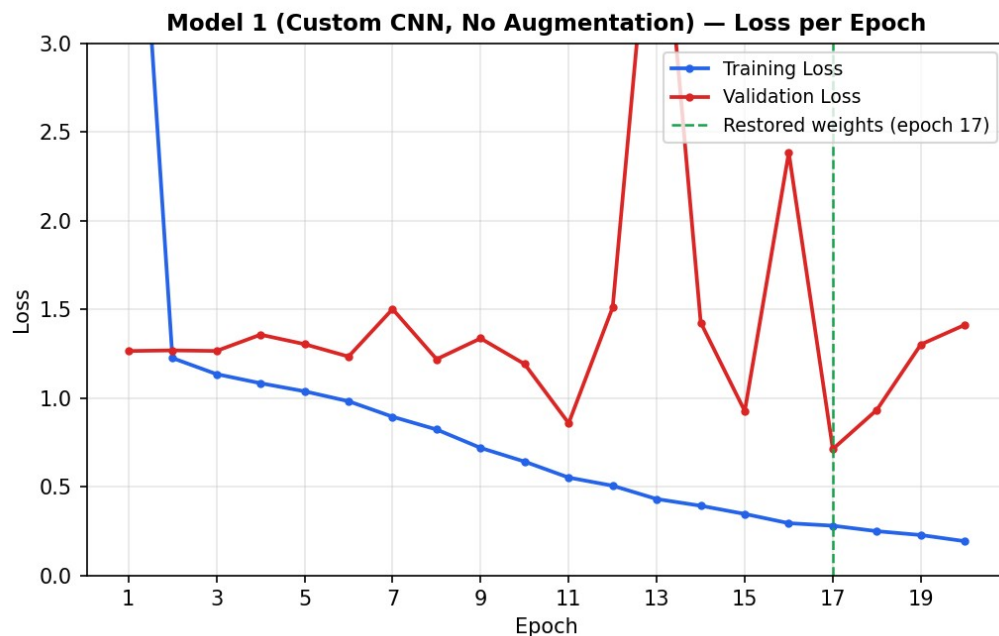


Figure 3. Model 1 — Training vs. validation loss per epoch.

The point where the two curves clearly diverge is around epoch 9-10: training accuracy is still climbing steadily past 70%, while validation accuracy remains stuck below 40% and validation loss starts becoming unstable rather than continuing to fall. From that point on, the model is increasingly fitting patterns

specific to the 1,821 training images rather than patterns that generalise — textbook overfitting, and unsurprising given how little data Model 1 had to work with. EarlyStopping (monitor='val\_loss', patience=8) restored the weights from epoch 17, which had the lowest validation loss (0.71) of any epoch, and those are the weights evaluated below.

## 5.4 Evaluation on the Held-Out Test Set

| Metric               | Value  | Interpretation                              |
|----------------------|--------|---------------------------------------------|
| Test Accuracy        | 78.36% | Correctly classified test images            |
| Precision (weighted) | 80.39% | Of predicted-positive labels, share correct |
| Recall (weighted)    | 78.36% | Of actual-positive labels, share detected   |
| F1-Score (weighted)  | 76.97% | Harmonic mean of precision & recall         |

The test accuracy of 78.36% and loss of 0.7123 match epoch 17's validation figures exactly, confirming that EarlyStopping's weight restoration worked as intended.

## 6. Model 2 — Custom CNN (With Augmentation)

Model 2 reuses the exact same architecture, compilation settings, and training configuration as Model 1 — the only variable that changed is the training data, which now includes the five augmentation techniques described in Section 4. This was a deliberate choice: keeping the architecture identical means any change in performance can be attributed to the augmented data rather than a change in model capacity.

### 6.1 Architecture (Identical to Model 1)

Because the architecture is unchanged, the layer-by-layer rationale in Section 5.1 applies here too and is not repeated. The parameter count is also identical, since augmentation changes the training data, not the model.

| Layer (type)                   | Output Shape         | Parameters |
|--------------------------------|----------------------|------------|
| conv2d (Conv2D, 32 filters)    | (None, 100, 100, 32) | 896        |
| batch_normalization            | (None, 100, 100, 32) | 128        |
| activation (ReLU)              | (None, 100, 100, 32) | 0          |
| max_pooling2d (2×2)            | (None, 50, 50, 32)   | 0          |
| conv2d_1 (Conv2D, 64 filters)  | (None, 50, 50, 64)   | 18,496     |
| batch_normalization_1          | (None, 50, 50, 64)   | 256        |
| activation_1 (ReLU)            | (None, 50, 50, 64)   | 0          |
| max_pooling2d_1 (2×2)          | (None, 25, 25, 64)   | 0          |
| conv2d_2 (Conv2D, 128 filters) | (None, 25, 25, 128)  | 73,856     |
| batch_normalization_2          | (None, 25, 25, 128)  | 512        |
| activation_2 (ReLU)            | (None, 25, 25, 128)  | 0          |
| max_pooling2d_2 (2×2)          | (None, 12, 12, 128)  | 0          |
| flatten                        | (None, 18432)        | 0          |
| dense (128, ReLU)              | (None, 128)          | 2,359,424  |
| dropout (rate = 0.2)           | (None, 128)          | 0          |
| dense_1 (4, Softmax)           | (None, 4)            | 516        |

### Parameter Summary

|                                 |                     |
|---------------------------------|---------------------|
| <b>Total parameters</b>         | 2,454,084 (9.36 MB) |
| <b>Trainable parameters</b>     | 2,453,636 (9.36 MB) |
| <b>Non-trainable parameters</b> | 448 (1.75 KB)       |

## 6.2 Training Configuration

Identical hyperparameters to Model 1: Adam optimiser, categorical cross-entropy, batch size 32, EarlyStopping on val\_loss with patience 8 and best-weights restoration.

| Hyperparameter       | Setting                   | Notes                                                        |
|----------------------|---------------------------|--------------------------------------------------------------|
| Optimiser            | Adam                      | Adaptive learning rate; good default for image CNNs          |
| Loss function        | Categorical Cross-Entropy | Standard for one-hot, mutually-exclusive multi-class targets |
| Batch size           | 32                        | Balances gradient stability and memory on Colab GPU          |
| Epochs (max)         | 20                        | Upper bound; EarlyStopping may cut this short                |
| Callback             | EarlyStopping             | monitor = 'val_loss'                                         |
| Patience             | 8 epochs                  | Training stops if val_loss does not improve for 8 epochs     |
| Restore best weights | True                      | Final model = weights from the best (lowest val_loss) epoch  |
| Validation data      | Held-out test split       | validation_data = (x_test, y_test)                           |

## 6.3 Training and Validation Curves

The difference from Model 1 is visible almost immediately. Validation accuracy tracks training accuracy far more closely from around epoch 5 onward, and by epoch 7 the model is already above 80% validation accuracy — something Model 1 never reached even once in 20 epochs. Validation loss trends downward overall rather than drifting upward, aside from one sharp spike at epoch 10 (2.22) that recovers by the following epoch.

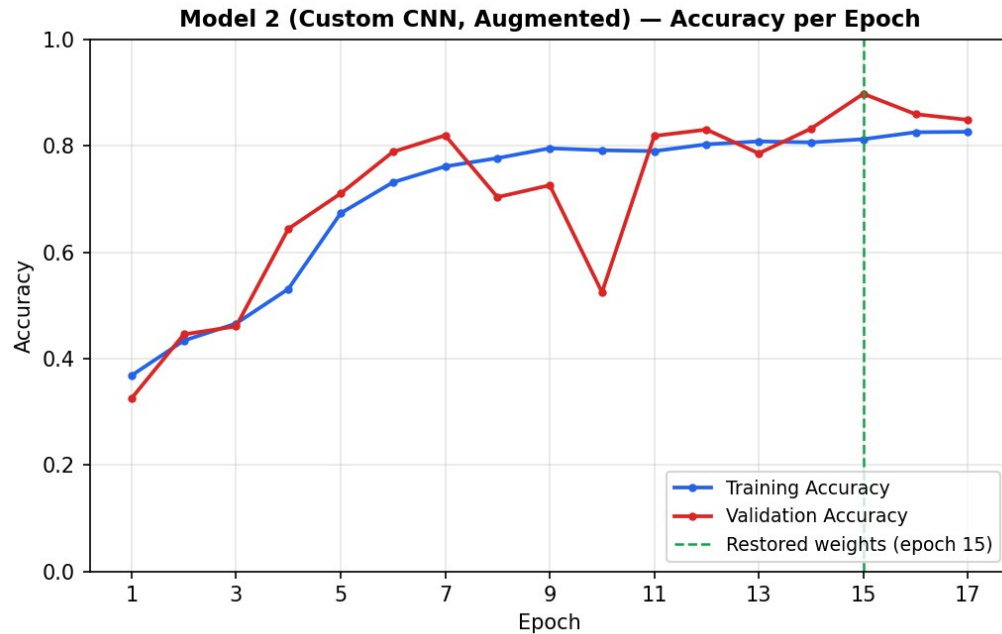


Figure 4. Model 2 — Training vs. validation accuracy per epoch.

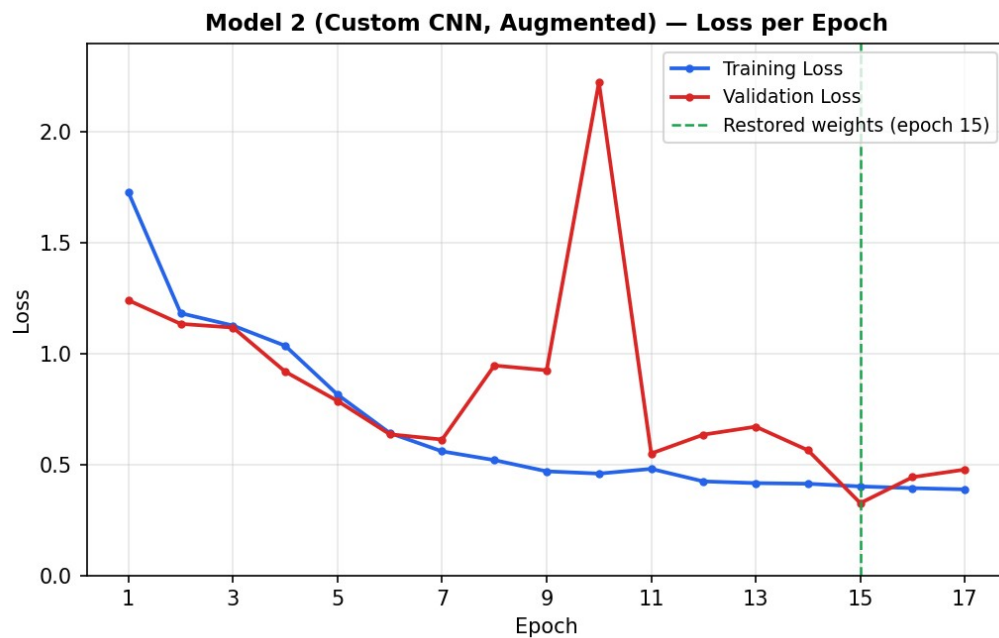


Figure 5. Model 2 — Training vs. validation loss per epoch.

EarlyStopping restored the weights from epoch 15, which had the lowest validation loss (0.326) of the run — and this matches the final test evaluation (accuracy 0.8980, loss 0.3260) exactly, again confirming the restoration behaved correctly.

## 6.4 Evaluation on the Held-Out Test Set

| Metric               | Value  | Interpretation                              |
|----------------------|--------|---------------------------------------------|
| Test Accuracy        | 89.80% | Correctly classified test images            |
| Precision (weighted) | 88.23% | Of predicted-positive labels, share correct |
| Recall (weighted)    | 89.80% | Of actual-positive labels, share detected   |
| F1-Score (weighted)  | 88.03% | Harmonic mean of precision & recall         |

## 7. Comparative Analysis — Model 1 vs. Model 2

- Test accuracy improved from 78.36% to 89.80% — an 11.4-point gain purely from training on augmented data with the same architecture.
- F1-score improved from 76.97% to 88.03%, an 11.1-point gain, showing the improvement is balanced across precision and recall rather than trading one off against the other.
- Overfitting starts far earlier and more severely in Model 1: the training/validation gap opens up around epoch 9–10 and never closes. In Model 2, the two curves stay close together for most of training, and the gap that does appear (e.g. the epoch-10 loss spike) recovers quickly rather than compounding.
- Because both models share the same architecture and parameter count (2,454,084 parameters in both cases), the entire performance gain can be attributed to the roughly 6× larger and more varied training set that augmentation produced — this is precisely the kind of controlled before/after comparison the assessment brief asks for.
- The practical takeaway: with only 1,821 original images, a 2.45-million-parameter network has more than enough capacity to memorise the training set rather than learn generalisable features, and augmentation is doing real, necessary work here rather than being a marginal improvement.

## 8. Model 3 — Transfer Learning (EfficientNetB0)

Model 3 is the transfer-learning stage of this project and, at the time of writing, its architecture has been defined and the base network prepared, but it has not yet been trained. It is included here to document the current state of the work rather than to report results that do not exist yet.

### 8.1 Architecture (Defined, Training Pending)

The base network is EfficientNetB0, loaded with its ImageNet-pretrained weights and with its top classification layer removed (`include_top=False`), taking a 224×224×3 input. The base is frozen (`base_model.trainable = False`), meaning its convolutional weights will not be updated during initial training — only the newly added classification head will learn. On top of the frozen base, the notebook adds:

- GlobalAveragePooling2D — collapses each of the base model's output feature maps to a single average value, which is a standard, low-parameter way to convert convolutional features into a flat vector (used here instead of Flatten, which would produce a much larger and more overfit-prone vector).
- Batch Normalisation — normalises the pooled features before they reach the dense layers, for the same stabilising reasons discussed in Section 5.1.
- Dense(256, ReLU) → Dropout(0.4) → Dense(128, ReLU) → Dropout(0.3) — a two-stage classification head with fairly aggressive dropout (40% and 30%), which makes sense given how easily a large pretrained network can overfit a dataset as small as this one.
- Dense(4, Softmax) — the same four-class output layer used in Models 1 and 2, for the same reason: mutually exclusive classes need a probability distribution that sums to 1.

Because training has not been executed for this model, no accuracy, loss, or evaluation metrics are reported here — the row for Model 3 in the comparison table below is intentionally left as "Pending." Before training this model, it would be worth revisiting the BGR/RGB channel-order point raised in Section 3, since it specifically matters for a pretrained ImageNet network in a way it did not for Models 1 and 2.

## 9. Overall Comparison Table

| Model                                        | Test Acc.      | Precision      | Recall         | F1-Score       | Parameters         |
|----------------------------------------------|----------------|----------------|----------------|----------------|--------------------|
| Model 1 — Custom CNN (No Aug.)               | <b>78.36%</b>  | <b>80.39%</b>  | <b>78.36%</b>  | <b>76.97%</b>  | <b>2,454,084</b>   |
| Model 2 — Custom CNN (Augmented)             | <b>89.80%</b>  | <b>88.23%</b>  | <b>89.80%</b>  | <b>88.03%</b>  | <b>2,454,084</b>   |
| Model 3 — Transfer Learning (EfficientNetB0) | <i>Pending</i> | <i>Pending</i> | <i>Pending</i> | <i>Pending</i> | <i>Base frozen</i> |

## 10. Conclusion and Next Steps

Between the two models that have been trained so far, augmentation made a substantial, measurable difference: Model 2 outperforms Model 1 by roughly 11 points on every metric, and — just as importantly — does so while showing far less overfitting, despite using an identical architecture. This confirms that the main bottleneck for Model 1 was the small size of the original dataset (1,821 images) relative to the model's capacity, rather than the architecture itself being unsuitable for the task.

The immediate next step is to complete Model 3: run the transfer-learning training (initially with the EfficientNetB0 base frozen, and potentially followed by a fine-tuning phase where some of the top base layers are unfrozen), and evaluate it on the same test split used for Models 1 and 2 so that the three-way comparison in Section 9 can be completed. It would also be worth correcting the BGR/RGB channel order noted in Section 3 before that training run, generating confusion matrices for all three models, and carrying out the misclassification analysis on the strongest model, all of which the assessment brief asks for and which have not yet been produced.

## 11. References

- Plant Pathology 2020 – FGVC7, Kaggle competition dataset: [kaggle.com/c/plant-pathology-2020-fgvc7](https://kaggle.com/c/plant-pathology-2020-fgvc7)
- Thapa, R. et al. (2020). The Plant Pathology 2020 challenge dataset to classify foliar disease of apples. [arXiv:2004.11958](https://arxiv.org/abs/2004.11958)
- TensorFlow / Keras documentation — [tensorflow.org/api\\_docs/python/tf/keras](https://tensorflow.org/api_docs/python/tf/keras)
- Tan, M. & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. [arXiv:1905.11946](https://arxiv.org/abs/1905.11946)